ESCOLA NAVAL

MARINHA PORTUGUESA

INTELIGÊNCIA ARTIFICIAL

Ano letivo 2026–2027



Docente: António Gonçalves

Como encontra o robô um caminho até ao objetivo?

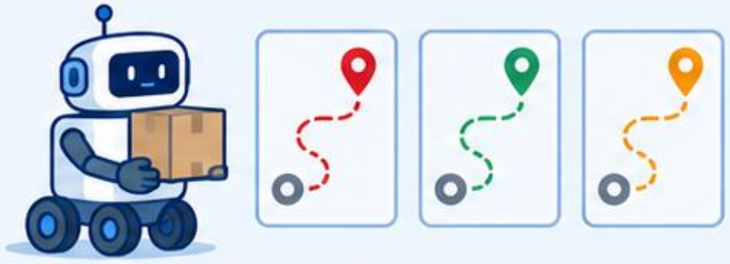
Na primeira semana vimos como o agente decide. Agora vamos perceber como constrói um plano antes de agir.

1 O que já sabemos

O robô tem uma tarefa, observa o ambiente e escolhe ações para atingir um objetivo.

Definimos requisitos e critérios para comparar percursos. Na arquitetura híbrida, o planeamento propõe caminhos até ao destino.

Nos exemplos anteriores, os percursos candidatos já eram apresentados.



2 A pergunta que falta

Como encontra o programa os percursos possíveis?

Conhecer o destino não indica que movimentos devem ser executados.

É necessário usar o mapa e as ações permitidas para construir uma sequência que leve o robô do início ao destino.



3 O que vamos estudar

Representar a situação inicial, as ações e os seus efeitos.

Explorar alternativas com diferentes estratégias de procura.

Verificar se o plano cumpre os requisitos e comparar o seu custo.

A solução procurada é uma sequência de ações.

O mesmo robô, uma nova pergunta



O robô parte de **B** com 8 unidades de energia e deve chegar a **D** com uma reserva mínima de 2 unidades.

Que sequência de movimentos permite cumprir a tarefa?

Para começar: mapa e custos conhecidos, movimentos com efeitos previsíveis e ambiente estável durante o planeamento.

A procura permite encontrar planos para um agente orientado para objetivos.

Formulação e procura não informada

O que vamos aprender nesta aula

1. **Formular o problema**
   Estado inicial, ações, transições, objetivo, restrições e custos.

2. **Representar a procura**
   Estados, nós e alternativas por explorar.

3. **Aplicar três estratégias**
   Procura em largura (BFS), em profundidade (DFS) e de custo uniforme (UCS).

4. **Comparar os resultados**
   Caminho encontrado, custo da solução, tempo e memória da procura.

5. **Verificar a solução e as garantias**
   Cumprimento dos requisitos e condições para encontrar uma solução ou uma solução de menor custo.

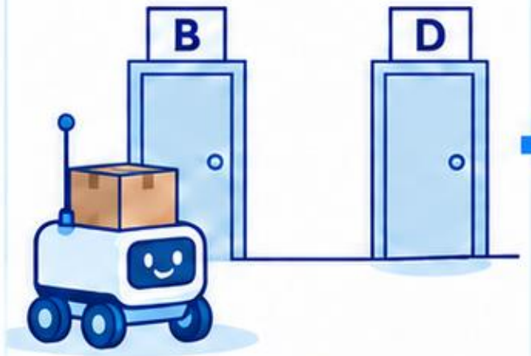
Começamos por transformar a tarefa do robô em requisitos concretos.

Do PEAS aos requisitos do problema

Na primeira semana usámos o PEAS para descrever a tarefa do robô. Agora usamos essa informação para preparar a procura de um plano.

Tarefa e informação inicial

Transportar uma encomenda de B para D.



Neste exemplo, o robô começa em B com **12 unidades** de energia.

S — Sensores

Fornecem informação para estimar a posição e medir a carga disponível.

Condições obrigatórias

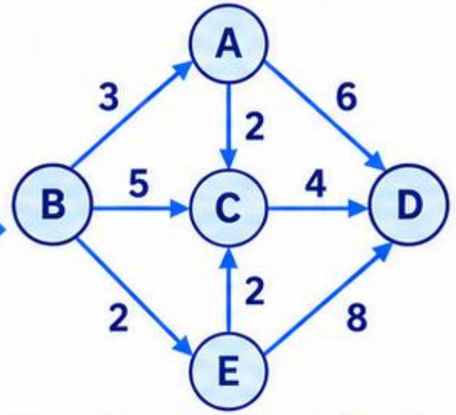
P — Medida de desempenho

- Chegar ao destino D.
- Usar apenas ligações permitidas.
- Ter energia para cada movimento.
- Terminar com uma reserva mínima de 2 unidades.

Um caminho válido cumpre todas estas condições.

Mapa e movimentos

E — Ambiente



Mapa fornecido: locais, ligações permitidas e energia consumida.

A — Atuadores

Os motores executam os movimentos escolhidos.

u.e. = unidade de energia (dados didáticos).

Critério de escolha

P — Medida de desempenho

Como comparar os caminhos válidos?

Entre os caminhos que cumprem todas as condições, queremos encontrar o que **consome menos energia.**



Minimizar a energia consumida.

O que verificar?

- O caminho chega a D?
- Todos os movimentos são permitidos e executáveis?
- A energia final é suficiente?
- Existe outro caminho válido que consuma menos?

No PEAS, **A** significa **atuadores.** Na procura, as **ações** são os movimentos que esses atuadores executam.

O PEAS descreve a tarefa do agente.

A formulação da procura define os estados e as ações para encontrar um plano.

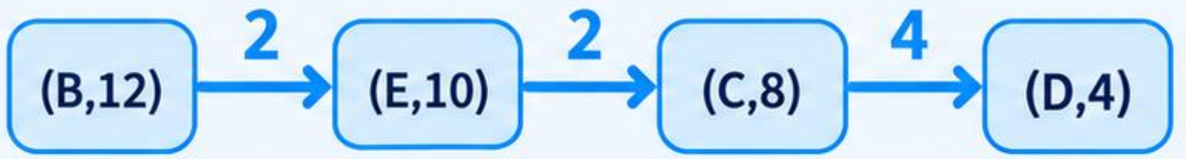
A seguir: estado inicial, ações, transições, objetivo e custos.

Como formular um problema de procura

Da tarefa do robô à descrição usada pelo algoritmo

$$P = (S, A, T, s_0, G, c)$$

Os seis componentes	Exemplo do robô
S	Estados: posição e energia, por exemplo (B,12)
$A(s)$	Ações: ligações permitidas com energia suficiente
$T(s,a)$	Transição: de (B,12) para (E,10)
s_0	Estado inicial: (B,12)
$G(s)$	Objetivo: estar em D com energia ≥ 2
$c(s,a,s')$	Custo da ação: B para E consome 2 unidades



Custo total: 2 + 2 + 4 = 8 unidades

Solução válida: chega a D com 4 unidades de energia.

Uma solução ótima é uma solução válida com menor custo. A validade, por si só, não garante que seja ótima.



Nesta aula: modelo conhecido, transições determinísticas, estados discretos e ambiente estático.
A formulação define o que o algoritmo pode fazer e como reconhecer uma solução.

Objetivo e solução ótima

QUESTÃO

Um robô começa em B com 12 unidades de energia. Deve chegar a D com pelo menos 2 unidades.
O estado é (posição, energia). Cada movimento consome energia; o custo total é a soma dos consumos.



Grupo A:

«Estar em D com energia suficiente.»



Grupo B:

«Estar em D, com pelo menos 2 unidades, tendo percorrido o caminho de menor custo.»



1 **Reformule a proposta do grupo A como uma condição precisa, que possa ser testada usando apenas o estado atual.**

2 **A proposta do grupo B mistura duas verificações. Identifique-as e escreva separadamente o objetivo e o critério de otimização.**

3 **O robô chega a (D, 3) por um percurso com todas as ações permitidas.**
A solução é válida? É necessariamente ótima? Justifique ambas as respostas.

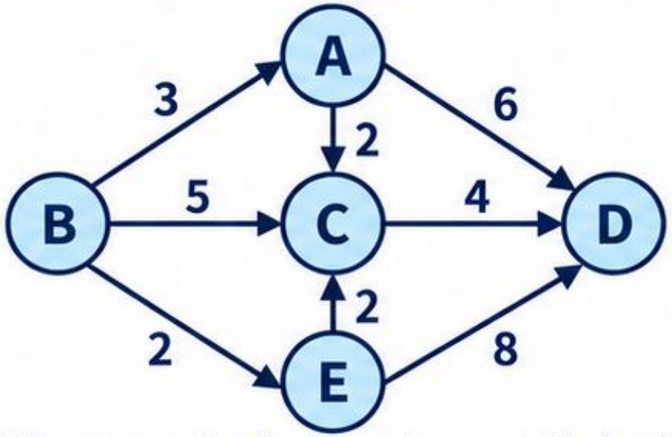
Estado, ação e transição

O grafo mostra as ligações; o estado indica a posição e a energia do robô.

Vamos aplicar o modelo: representar a situação do robô e calcular o que muda quando se desloca.

1. Grafo e estado do robô

O robô só pode deslocar-se pelas ligações indicadas. Cada movimento consome energia.



Números nas ligações: energia consumida (u.e.).

Estado = (posição, energia)

$$s = (p, e)$$

Estado inicial: (B, 12)

O robô está em B e dispõe de 12 unidades de energia.

Os pontos do grafo representam locais.
O estado inclui também a energia.

2. Ação e transição

A ação $B \to C$ muda a posição e reduz a energia disponível.



(B, 12)
Antes da ação

(C, 7)
Depois da ação

Energia final: $12 - 5 = 7$

A ação é permitida porque:

- Existe a ligação $B \to C$.
- A energia disponível é suficiente: $12 \geq 5$.

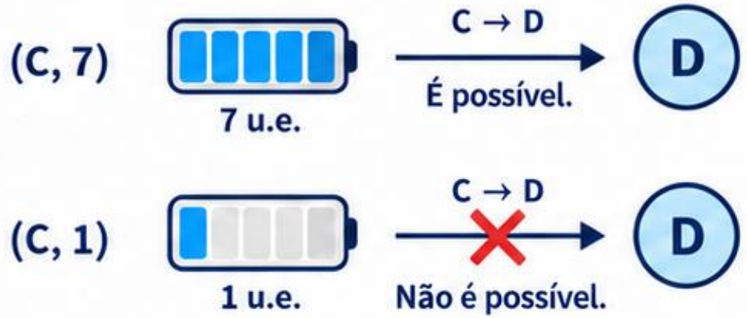
A transição calcula o estado resultante da ação.

$$T((B, 12), B \to C) = (C, 7)$$

3. A posição não chega

No mesmo local, energias diferentes podem permitir ações diferentes.

Estado	Pode executar C → D?
$(C, 7)$	**Sim:** tem 7 e precisa de 4.
$(C, 1)$	**Não:** tem 1 e precisa de 4.



A energia faz parte do estado porque influencia os movimentos que o robô pode executar.

$(C, 1)$ é uma situação hipotética para comparação; não resulta da transição mostrada.

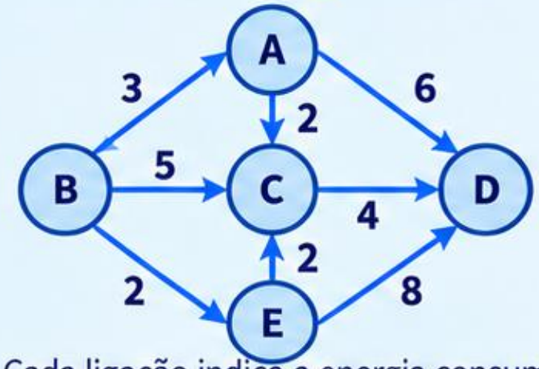
O grafo indica as ligações possíveis. A posição e a energia determinam quais as ações que o robô pode executar.

Objetivo, restrições e custo

Cumprir os requisitos e escolher a solução de menor custo

Já sabemos calcular o resultado de uma ação. Agora vamos verificar se um percurso resolve o problema e comparar o seu custo.

1. O mesmo problema





Cada ligação indica a energia consumida (u.e.).

Estado inicial: (*B*, 12)
O robô parte de B com 12 unidades de energia.

Só pode usar ligações existentes e executar movimentos para os quais tenha energia suficiente.

2. Quando cumpre o objetivo?

A entrega está concluída quando o robô chega a D e conserva, pelo menos, 2 unidades de energia.

Posição final = D **E** **Energia final ≥ 2**

$$G(p, e) = (p = D) \wedge (e \geq 2)$$

As duas condições têm de ser cumpridas.

Um estado intermédio pode ser válido sem satisfazer o objetivo.

3. Como comparamos os percursos?

O custo de um percurso é a soma da energia consumida em todas as suas ações.

Custo total = soma dos custos das ações

Energia final = 12 – custo total

Entre os percursos que cumprem os requisitos, procuramos o que consome menos energia.

Importante:
Neste exemplo, o custo é medido em unidades de energia (u.e.).

Dois percursos admissíveis, custos diferentes

Percurso	N.º de ações	Custo total (u.e.)	Energia final (u.e.)	Admissível?	Comparação
B → C → D	2	5 + 4 = 9	12 – 9 = 3	Sim	Cumpre os requisitos
B → E → C → D	**3**	**2 + 2 + 4 = 8**	**12 – 8 = 4**	**Sim**	**Menor custo: solução ótima**

Escolhemos **B → E → C → D** porque consome menos energia (8 < 9) e mantém a reserva exigida.

Uma solução admissível cumpre os requisitos. Uma solução ótima tem o menor custo entre todas as soluções admissíveis.

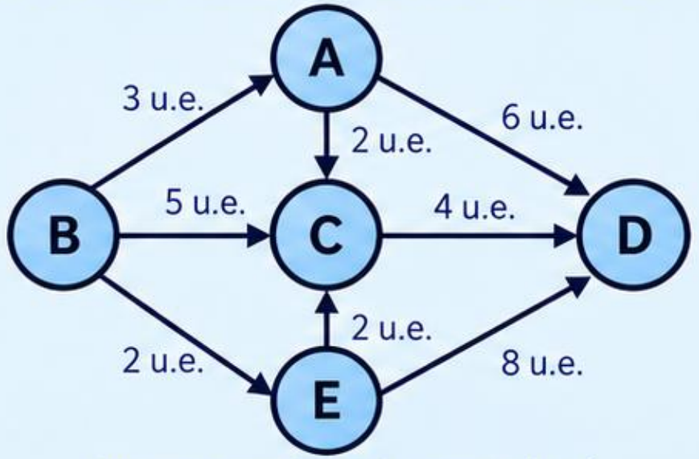
Do estado ao nó de procura

O estado descreve a situação; o nó guarda também como chegámos até ela.

Mantemos o mesmo grafo e o estado inicial (B, 12). Durante a procura, guardamos os estados alcançados e a informação sobre cada percurso.

1. O grafo de locais

O mapa indica os locais, as ligações permitidas e o consumo de energia de cada movimento.



Custos em unidades de energia (u.e.).

Estado inicial: (B, 12)
Posição B; energia disponível 12.

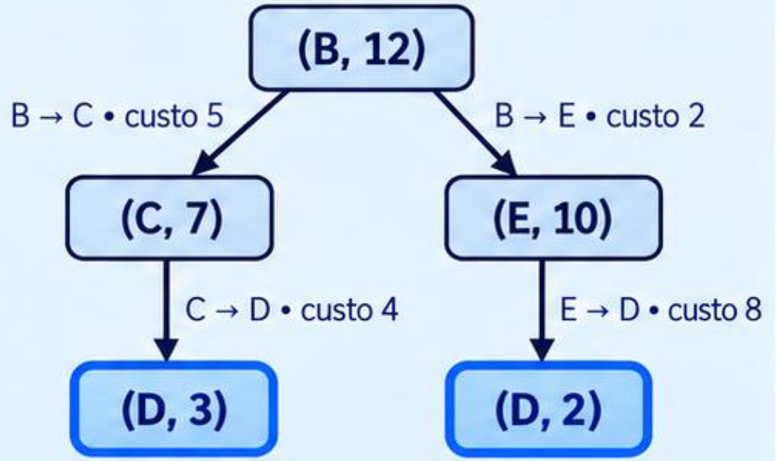
Estado = (posição, energia)
Exemplos: (C, 7), (D, 3) e (D, 2).

Os pontos deste mapa são locais; o estado do robô inclui também a energia.

2. A árvore de procura

Excerto da árvore: dois percursos.

Cada nó representa um estado alcançado por um determinado caminho.



Exemplo detalhado no bloco 3

Mesmo local, estados diferentes
(D, 3) e (D, 2) têm a mesma posição, mas energias diferentes.

3. O que guarda um nó?

Exemplo: nó alcançado pelo percurso B → C → D.

Campo	Valor neste exemplo
Estado	(D, 3)
Pai	Nó com estado (C, 7)
Ação	C → D
Custo acumulado, *g*	5 + 4 = 9 u.e.
Profundidade, d	2 ações

O custo soma a energia consumida desde o início.
A profundidade conta as ações executadas.

nó = (estado, pai, ação, g, d)

Como recuperamos o percurso?

Seguimos os nós pai: (D, 3) → (C, 7) → (B, 12).
Invertendo a ordem, obtemos o percurso: B → C → D.
(Esta sequência é obtida seguindo os nós pai, não são movimentos no grafo.)

O estado descreve a situação do robô. O nó de procura guarda também o caminho que permitiu alcançá-la.

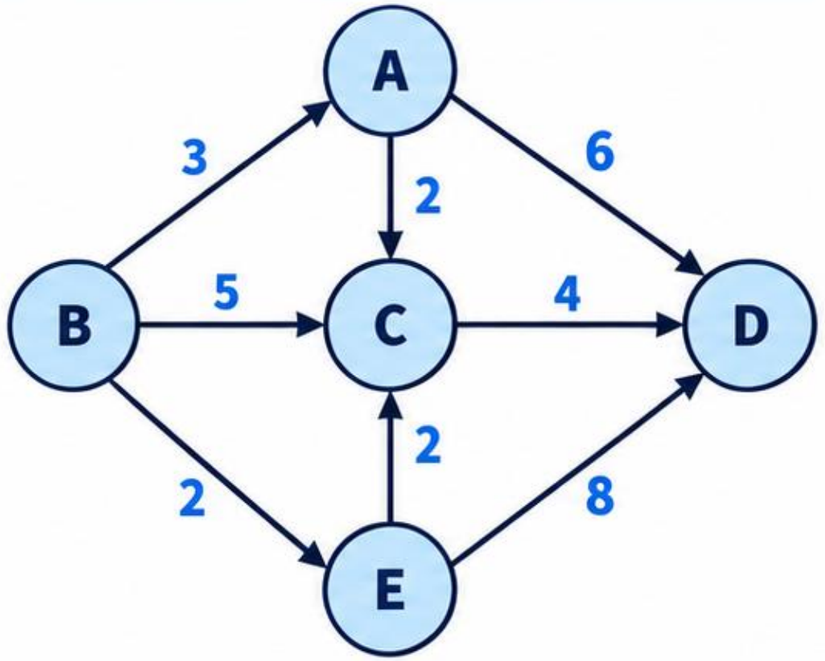
Fronteira e ciclo geral de procura

O mapa mostra as ligações. A fronteira guarda os nós que aguardam seleção.

1. Mapa do problema

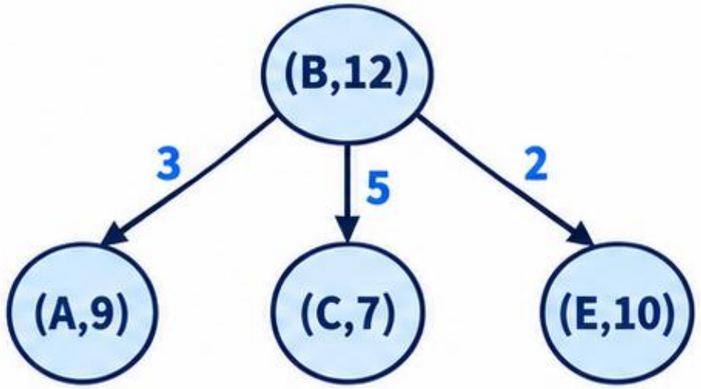
Estado inicial:
(B,12)

Objetivo:
D com energia ≥ 2



O mapa mostra as ligações disponíveis.

2. Evolução da fronteira



1	**Inicializar:**	[(B,12)]
2	**Retirar (B,12):**	[]
3	**Testar objetivo:**	[]
4	Expandir e inserir:	[(A,9), (C,7), (E,10)]

A fronteira fica temporariamente vazia após a retirada. O nó é processado antes da próxima verificação.

A ordem de seleção depende da estratégia.

3. Ciclo de procura

1. **Fronteira vazia?**
   Terminar sem solução.
2. **Retirar um nó da fronteira.**
3. **Objetivo atingido?**
   Devolver o caminho.
4. **Caso contrário, expandir e atualizar a fronteira.**
5. **Repetir.**

Gerar: criar um sucessor.
Retirar: selecionar um nó da fronteira.
Expandir: gerar sucessores válidos.

O grafo descreve o problema. A fronteira mostra os nós disponíveis para analisar a seguir.

Mesmo estado, mesmo nó?

QUESTÃO

Um robô começa em B com 12 unidades de energia.
O estado é (posição, energia). Cada ação consome a energia indicada; não há recarga.



Percurso 1: B → C



Percurso 2: B → A → C



O custo acumulado soma os consumos. A profundidade conta as ações.

Aluno A:
«Caminhos diferentes levam necessariamente a estados diferentes.»

Aluno B:
«Se o estado final é igual, os nós de procura são iguais em todos os campos.»

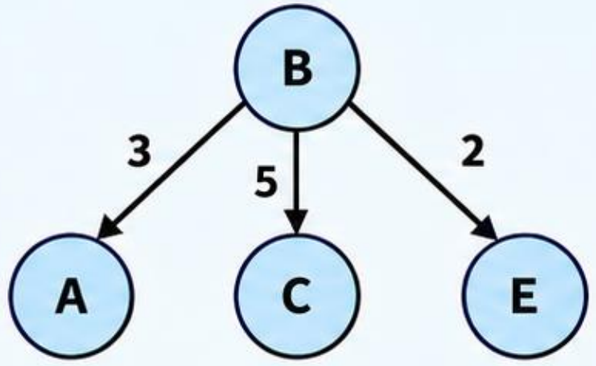
1. Calcule a energia final de cada percurso e indique se chegam ao mesmo estado.
2. Compare os dois nós em C: estado, pai, custo acumulado e profundidade.
3. Avalie as duas afirmações. Justifique por que um mesmo estado pode aparecer em nós de procura diferentes.

Três regras para selecionar o próximo nó

Procura em largura (BFS), em profundidade (DFS) e de custo uniforme (UCS)

Mesmo mapa, modelo simplificado: para comparar as **estratégias**, o **estado** é apenas o local e mantemos os custos.
A reserva de energia será verificada no percurso encontrado: 12 – custo ≥ 2.

Ligações a partir de B



B é o estado inicial: g(B) = 0.

Depois de retirar e expandir B, inserimos os sucessores pela ordem A, C, E.

Sucessor	Custo da ação desde B
A	3
C	5
E	2

Os números nas ligações indicam o custo de cada ação. Como B é o início, g(B) = 0.

BFS — Largura

Primeiro a entrar, primeiro a sair

Usa uma fila e seleciona os nós por menor profundidade. Entre estes sucessores, retira o primeiro inserido.



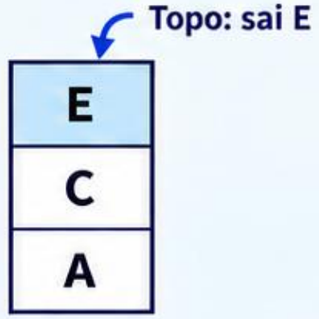
Próximo nó a retirar: A

A foi o primeiro a entrar na fila.

DFS — Profundidade

Último a entrar, primeiro a sair

Usa uma pilha. Com a inserção A, C, E, o nó E fica no topo.



Próximo nó a retirar: E

E foi o último a entrar na pilha.
A ordem de inserção influencia o ramo seguido.

UCS — Custo uniforme

Menor custo acumulado primeiro

Retira, de toda a fronteira, o nó com menor custo acumulado desde o início.

g(filho) = g(pai) + custo da ação

Nó	Custo acumulado g
E	0 + 2 = 2
A	0 + 3 = 3
C	0 + 5 = 5

Primeiro nó a retirar: E

No passo seguinte

Ao expandir E: C fica com g = 2 + 2 = 4 e D com g = 2 + 8 = 10.

A: 3	C: 4	D: 10

A UCS retira A, que já estava na fronteira.

Mesma escolha, razões diferentes

DFS e UCS escolhem **E** neste momento. A DFS considera a ordem de inserção; a UCS considera o custo acumulado.
O controlo de estados repetidos será explicado nos exemplos de cada algoritmo.

A estratégia determina a ordem de retirada da fronteira; não altera os custos das ligações.

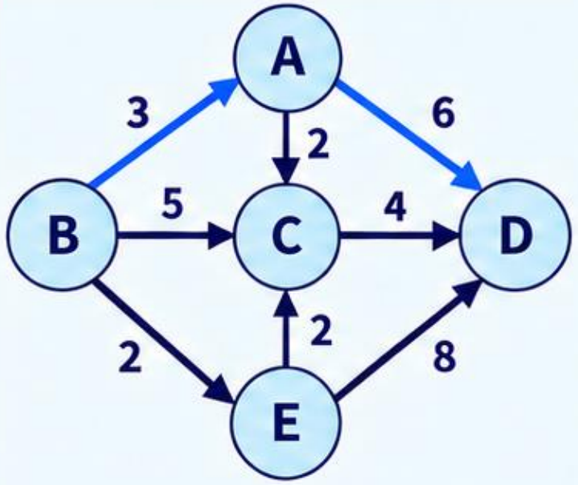
BFS: procura em largura

Explorar por níveis: primeiro 1 ação, depois 2 ações

1. Do grafo à árvore de descoberta

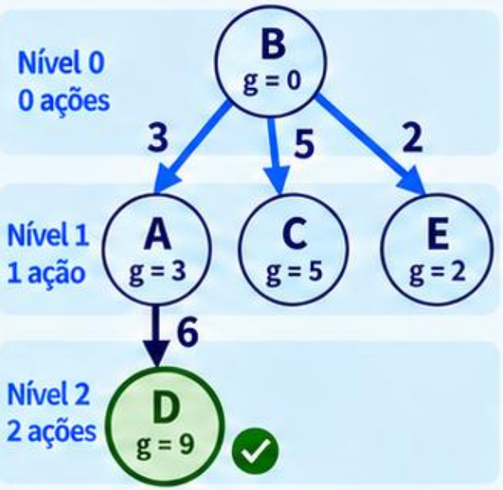
Partimos de **B** e procuramos D. O estado é apenas o local.

Grafo completo



Nas ligações: custo da ação.

Árvore de descoberta da BFS



Objetivo encontrado
ao retirar D da fila

A, C e E são descobertos ao expandir **B**. **D** é descoberto ao expandir **A**.
C e D não são reinseridos quando voltam a surgir.
A, C e E sãoretirados antes de **D**. Os níveis contam ações, não custos.

2. Fila e estados alcançados

Fila FIFO: primeiro a entrar, primeiro a sair.

Depois de expandir B



Alcançados: {B, A, C, E}

Retirar e expandir A

- **C** — já alcançado: não reinserir.
- **D** — novo: inserir e guardar pai(D) = A.



Alcançados: {B, A, C, E, D }

Retirar da fila não apaga o estado dos alcançados.
D guarda a retirada de C e E.

3. Algoritmo em linguagem simples

1. Criar o nó inicial com g = 0, colocá-lo na fila e marcar o seu estado como alcançado.
2. **Enquanto** a fila não estiver vazia:
   - Retirar o primeiro nó.
   - Se o seu estado satisfaz o objetivo, devolver o caminho.
   - Gerar sucessores por ordem alfabética.
   - Para cada sucessor cujo estado ainda não esteja nos alcançados:
     - Guardar o pai e a ação.
     - Calcular g(filho) = g(pai) + custo da ação.
     - Marcar o estado como alcançado.
     - Inserir o nó no fim da fila.
3. Se a fila se esgotar, devolver falha.

g regista o custo do caminho; não altera a ordem FIFO.
A fila guarda nós; os alcançados guardam estados.

O que garante?

- Se existir uma solução e a ramificação for finita, a BFS encontra uma solução com o menor número de ações.
- Com custos iguais e positivos, essa solução também tem o menor custo.

A BFS explora por níveis. A fila garante que os nós menos profundos são retirados primeiro.

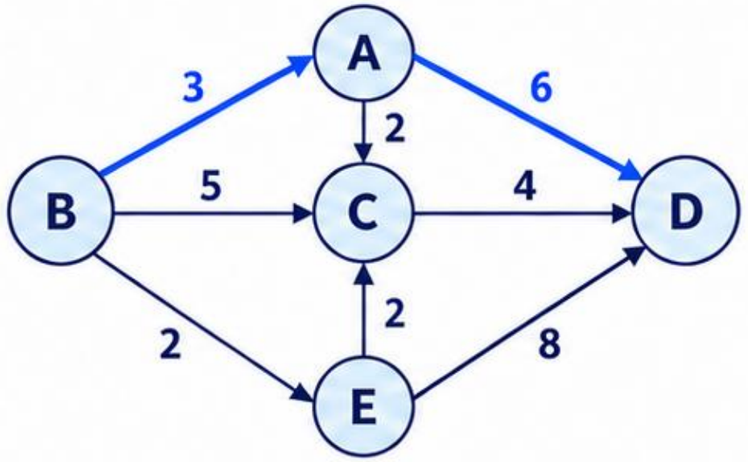
BFS: execução passo a passo

Seguir a fila e compreender o caminho encontrado

Início: B • Objetivo: D • Sucessores por ordem alfabética • Marcar ao inserir • Testar o objetivo ao retirar

1. O grafo completo

O mesmo exemplo: o estado é apenas o local.
Os números representam custos.



Nas setas: custo de cada ação.

O custo das ligações não determina a ordem da fila.
Guardamos o pai de cada estado quando o inserimos pela primeira vez.

2. Evolução da fila

O primeiro nó da fila está à esquerda.

Operação	Fila	d	Explicação
Inicializar	[B]	—	Inserir o nó inicial.
Retirar e expandir B	[A, C, E]	0	Inserir A, C e E.
Retirar e expandir A	[C, E, D]	1	C já foi alcançado. Inserir D, com pai A.
Retirar e expandir C	[E, D]	1	D já foi alcançado; não voltar a inserir.
Retirar e expandir E	[D]	1	C e D já foram alcançados.
Retirar D	[ ]	2	Objetivo encontrado. D não é expandido.

D entra na fila ao expandir A, mas a procura só termina quando D é retirado.

A, C e E são retirados antes de D porque estão a menor profundidade.

Concluímos o nível 1 antes de retirar D, no nível 2.

3. Resultado

Caminho encontrado
B → A → D

Custo total
3 + 6 = 9

Profundidade
2 ações

Ordem de retirada: B, A, C, E, D.

5 nós retirados

4 nós expandidos

Foram expandidos B, A, C e E.
D foi retirado e reconhecido como objetivo.

Menos ações não significa menor custo

B → **A** → D: 2 ações, custo 9 — caminho encontrado pela BFS.
B → **E** → **C** → D: 3 ações, custo 2 + 2 + 4 = 8 — mais barato, mas mais longo em número de ações.

A BFS dá prioridade ao número de ações, não ao custo acumulado.

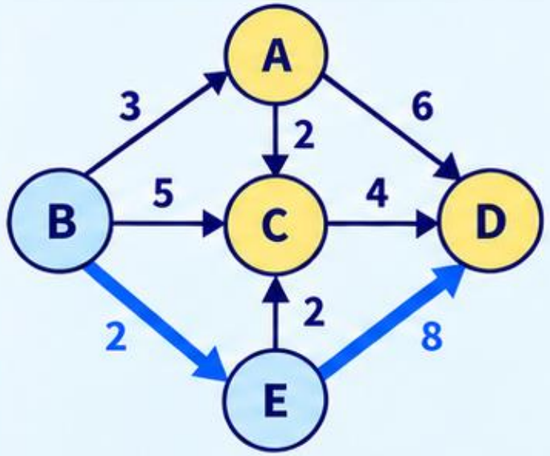
DFS: procura em profundidade

Aprofundar um ramo antes de explorar alternativas

1. Do grafo à árvore de descoberta

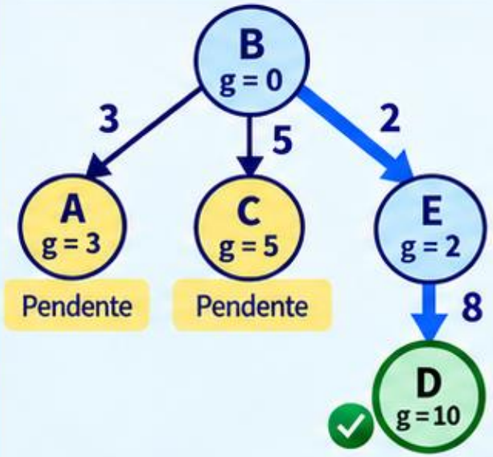
Partimos de B e procuramos D. O estado é apenas o local.

Grafo completo



Números nas ligações = custos das ações.

Árvore de descoberta da DFS



g = custo acumulado desde B.
A DFS não usa g para selecionar nós.

Objetivo encontrado ao retirar D da pilha

Com inserção A, C, E, retiramos E primeiro e seguimos B → E → D.
C já foi alcançado ao expandir B; não é reinserido ao expandir E.
A e C permanecem na pilha quando retiramos D e terminamos.

2. Pilha e estados alcançados

Pilha LIFO: último a entrar, primeiro a sair.

Depois de expandir B



Retirar / inserir no topo

Alcançados: {B, A, C, E}

Retirar e expandir E

C — já alcançado: não reinserir.

D — novo: guardar pai(D) = E.



Alcançados: {B, A, C, E, D}

Retirar da pilha não apaga o estado dos alcançados.
D sai a seguir; A e C ficam pendentes.

3. Algoritmo em linguagem simples

1. Criar o nó inicial com g = 0, colocá-lo na pilha e marcar o seu estado como alcançado.
2. Enquanto a pilha não estiver vazia:
   - Retirar o nó do topo.
   - Se o seu estado satisfaz o objetivo, devolver o caminho.
   - Gerar sucessores por ordem alfabética.
   - Para cada sucessor cujo estado ainda não esteja nos alcançados:
     - Guardar o pai e a ação.
     - Calcular g(filho) = g(pai) + custo da ação.
     - Marcar o estado como alcançado.
     - Inserir o nó no topo da pilha.
3. Se a pilha se esgotar, devolver falha.

g regista o custo do caminho; não altera a ordem LIFO.
A pilha guarda nós; os alcançados guardam estados.

O que garante?

- Num grafo finito, com controlo de estados repetidos, encontra uma solução se existir.
- Num espaço infinito, pode prosseguir indefinidamente num ramo.
- Não garante o menor número de ações nem o menor custo.

A DFS aprofunda um ramo. A pilha guarda as alternativas que poderão ser retomadas.

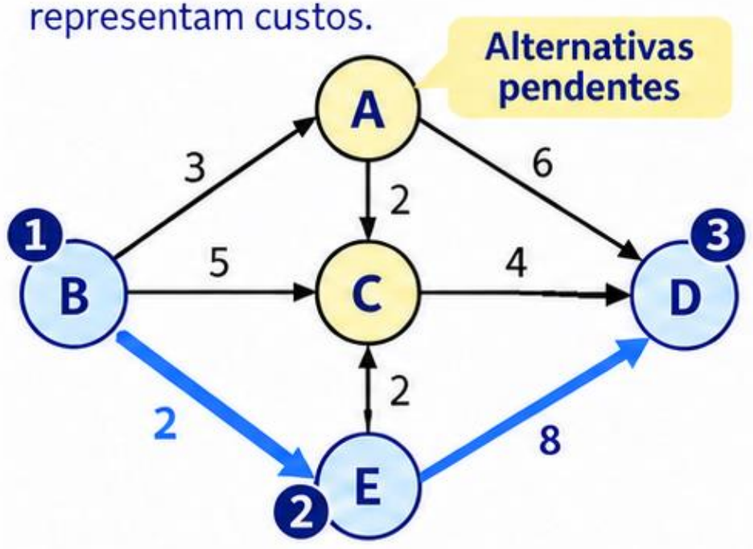
DFS: execução passo a passo

Aprofundar o ramo B → E → D e manter alternativas na pilha

Início: B • Objetivo: D • Inserção alfabética • Marcar ao inserir • Testar ao retirar

1. O mesmo grafo

Estado: apenas o local. Os números representam custos.



Os custos não determinam a ordem da pilha.

2. Evolução da pilha

Topo à direita: o último elemento é retirado primeiro.

Operação	Pilha	Explicação
Inicializar	[B]	Inserir B.
Retirar e expandir B	[A, C, E]	Inserir A, C e E. E fica no topo.
Retirar e expandir E	[A, C, D]	C já foi alcançado. Inserir D, com pai E.
Retirar D	[A, C]	Objetivo encontrado. D não é expandido.

Pilha no fim: alternativas não exploradas. [A] [C] ← Topo

- A DFS segue **B → E → D** antes de retomar **A** ou **C**. Como **D** é o objetivo, termina sem precisar de regressar.
- **C** já tinha sido alcançado ao expandir B; não é reinserido a partir de E.

3. Resultado

Caminho encontrado: B → E → D

Custo total: 2 + 8 = 10

Profundidade: 2 ações

Ordem de retirada: B, E, D

3 nós retirados | **2 nós expandidos**

Apenas **B e E** foram expandidos. **D** não foi expandido.

O resultado depende da ordem de inserção

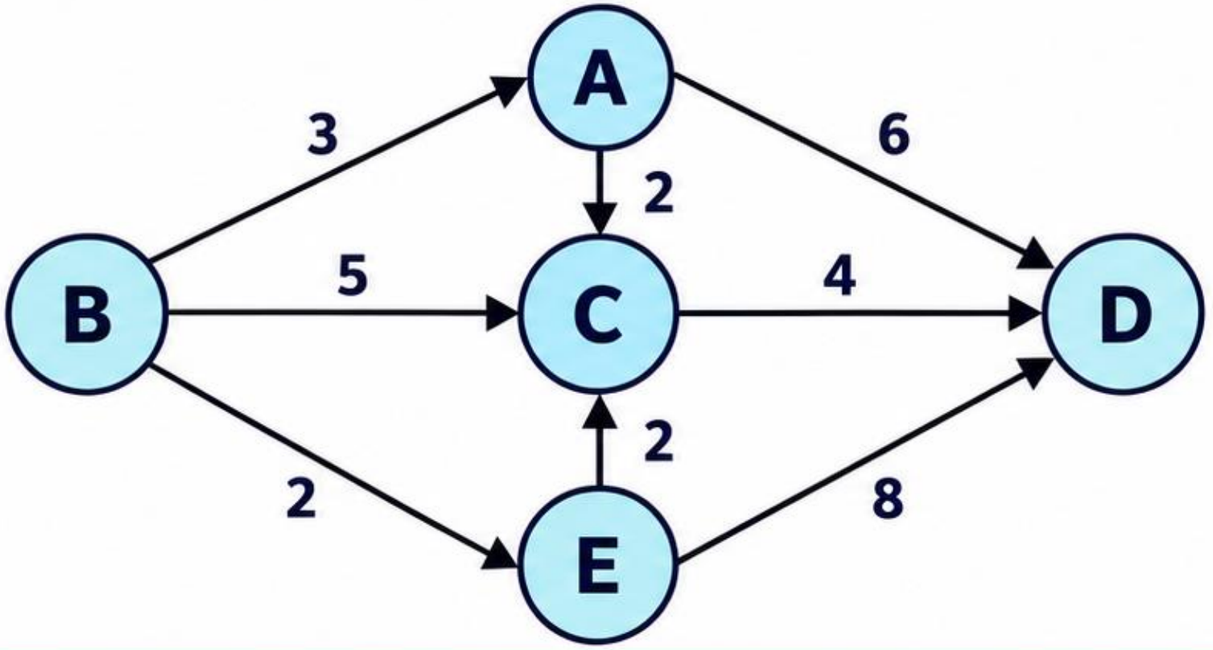
- Com inserção alfabética **A, C, E**, a DFS segue **E** primeiro e encontra **B → E → D**.
- Neste exemplo, encontra uma solução com 2 ações, mas isso não é uma garantia geral.
- Existe um caminho mais barato: **B → E → C → D**, com custo 8.

A primeira solução encontrada pela DFS não é necessariamente a de menor custo.

Alterar a ordem muda a solução?

QUESTÃO

Início: B. Objetivo: D. Os números nos arcos indicam custos.



Alteração: ao expandir B, inserir os sucessores pela ordem **E, C, A** (em vez de A, C, E).

Nos restantes nós, inserir por ordem alfabética.
Marcar cada estado quando entra na fronteira;
não o reinserir nem mudar o seu pai.
Testar o objetivo ao retirar.

Na **BFS:** fila **FIFO.** Na **DFS:** pilha **LIFO,** com o topo à direita.

1 Qual será o próximo nó retirado pela BFS e pela DFS, depois de expandir B?

2 Que caminho até D encontra cada algoritmo? Calcule a profundidade e o custo.

3 A mudança de ordem retira à BFS a garantia de encontrar o menor número de ações? Justifique.

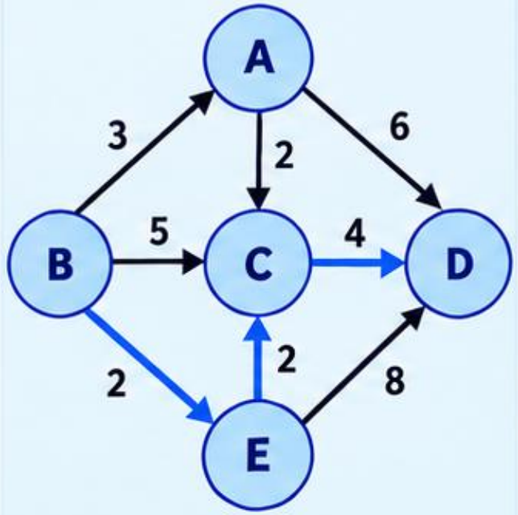
UCS: procura de custo uniforme

Escolher o menor custo acumulado desde o início

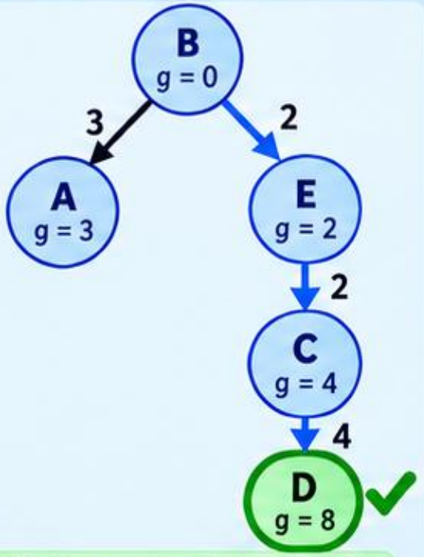
1. Do grafo aos melhores caminhos

Partimos de **B** e procuramos D. O estado é apenas o local.

Grafo completo



Árvore final de melhores caminhos



Objetivo encontrado ao retirar D.

Nas setas: custo da ação. **Nos nós**: custo acumulado *g*.

g(filho) = g(pai) + custo da ação

Caminho final: B → E → C → D. **Custo:** 2 + 2 + 4 = 8.
A árvore mostra os pais finais, após melhorar os caminhos.

2. Uma fila de prioridade

Retiramos o menor *g* de toda a fronteira.

Depois de expandir B:

Inserir ou atualizar por prioridade

Retirar menor g

E	A	C
g = 2	g = 3	g = 5

Representação ordenada por g; **não é uma *fila FIFO*.**

Próximo nó a retirar: E

Melhorar caminhos já encontrados

Ao expandir E, C melhora: 5 → 2 + 2 = 4.
O pai de C passa a ser E.

A	C	D
g = 3	g = 4	g = 10

Agora retiramos A, que já estava na fronteira.

A prioridade depende do custo total do caminho, não apenas da última ação.

3. Algoritmo em linguagem simples

1. Inserir o nó inicial com g = 0 e registar esse custo.
2. Enquanto a fronteira não estiver vazia:
   - Retirar o nó com menor g.
   - Se satisfaz o objetivo, devolver o caminho.
   - Gerar sucessores e calcular os novos custos.
   - Se o estado é novo ou o custo melhora: atualizar custo, pai, ação e prioridade.
3. Se a fronteira se esgotar, devolver falha.

Neste exemplo, substituímos a entrada anterior quando o custo melhora.

Em empate de custo, sai o primeiro inserido.

O que garante?

Num grafo finito, com custos não negativos e atualização correta dos caminhos, encontra uma solução de menor custo, se existir.

Em espaços infinitos, exigem-se condições adicionais: ramificação finita e custo por ação ≥ ε > 0.

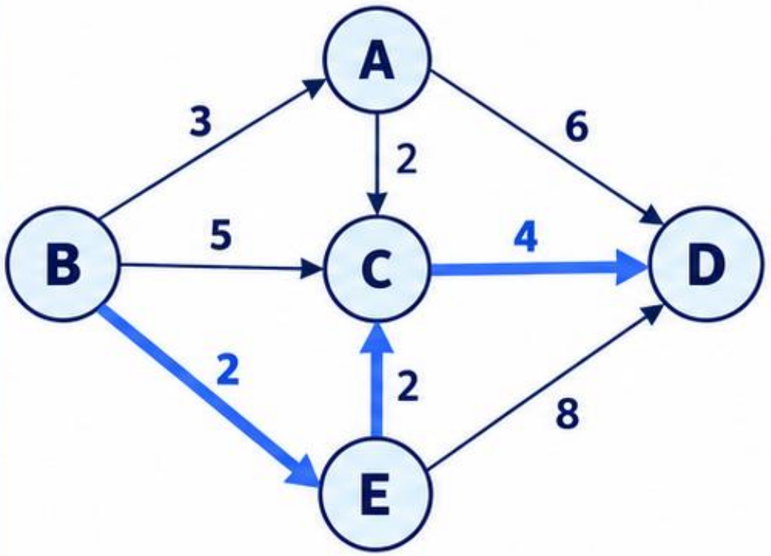
O objetivo é aceite ao retirar o nó de menor custo, não quando é gerado pela primeira vez.

UCS: execução passs o a passo

Atualizar caminhos até retirar o objetivo com o menor custo

Início: B • Objetivo: D • Selecionar menor g • Atualizar se o custo melhorar • Testar ao retirar

1. O mesmo grafo



Estado: apenas o local. *g* é o custo acumulado desde B.

Um caminho mais barato atualiza o custo, o pai e a prioridade do nó.

2. Evolução da fila de prioridade

Menor *g* à esquerda. Escrevemos estado:custo.

Operação	Fronteira	Atualizações
Inicializar	[B:0]	Custo inicial 0.
Retirar e expandir B	[E:2, A:3, C:5]	Inserir os três sucessores.
Retirar e expandir E	[A:3, C:4, D:10]	C melhora de 5 para 4; inserir D com 10.
Retirar e expandir A	[C:4, D:9]	Até C: 3+2=5, não melhora 4. D melhora de 10 para 9.
Retirar e expandir C	[D:8]	D melhora de 9 para 8; pai passa a C.
Retirar D	[]	Objetivo encontrado; não expandir D.

D é gerado com custo 10. A procura continua e melhora esse custo para 9 e depois 8.

A tabela mostra apenas as entradas atuais; as anteriores são substituídas.

3. Resultado

Caminho encontrado:
B → E → C → D

Custo mínimo:
2 + 2 + 4 = 8

Profundidade:
3 ações

Ordem de retirada:
B, E, A, C, D

5 nós retirados

4 nós expandidos

Pais finais:
D tem pai C; C tem pai E; E tem pai B.

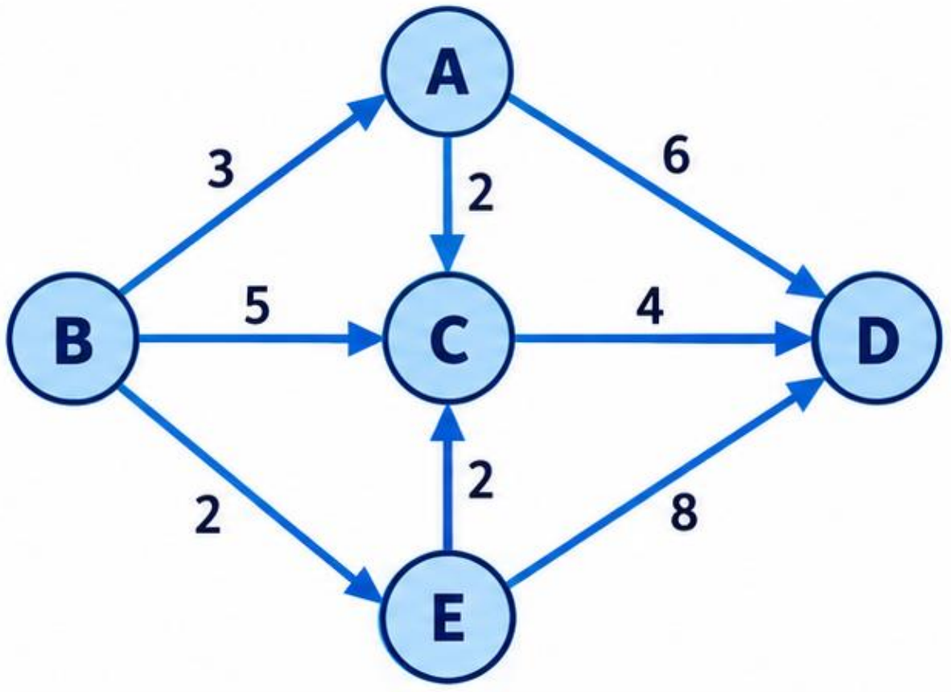
Porque não terminamos ao gerar D?

O primeiro caminho até D custa 10. Só aceitamos o objetivo quando é retirado como o nó de menor custo da fronteira.
Com os custos não negativos deste grafo, o caminho devolvido tem o menor custo.

A UCS encontra custo 8 com 3 ações. A BFS encontra 2 ações com custo 9: os critérios são diferentes.

Comparar BFS, DFS e UCS

Três estratégias, o mesmo grafo e diferentes regras de seleção



Partida: B com 12 unidades de energia

Objetivo: chegar a D com energia ≥ 2

Resultados desta execução

Estratégia	Próximo nó	Caminho	Ações	Custo	Energia final
BFS	mais antigo	B → A → D	2	9	3
DFS	mais recente	B → E → D	2	10	2
UCS	menor custo acumulado	B → E → C → D	3	8	4

Ordem de retirada

Estratégia	Ordem	Retirados	Expandidos
BFS	B, A, C, E, D	5	4
DFS	B, E, D	3	2
UCS	B, E, A, C, D	5	4

Retirado: selecionado da fronteira. **Expandido:** gerou sucessores.
D é retirado, mas não expandido.

Objetivo testado ao retirar; sucessores por ordem alfabética; **BFS** e **DFS** marcam **ao inserir**; **UCS** atualiza a fronteira se encontrar custo menor.
Para controlar repetições usa-se o **local**; a energia final calcula-se pelo custo do caminho.

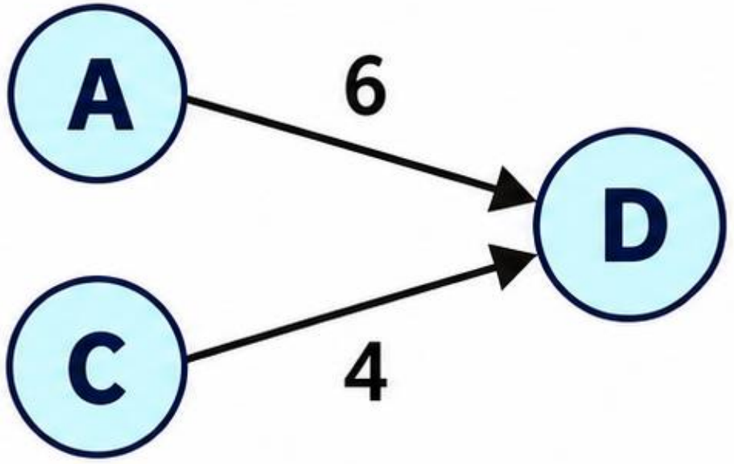
Menos ações não implica menor custo.

Neste exemplo, a UCS encontrou o menor custo; a DFS retirou menos nós.

UCS: encontrar o objetivo basta para terminar?

QUESTÃO

Numa execução da UCS, D é o objetivo.
Os custos dos arcos são não negativos.



OPEN = [A:3, C:4, D:10]

A:3 significa que o melhor caminho conhecido desde o início até A custa 3.

Nas próximas expansões, considerar apenas estas duas ligações. D não tem sucessores.

Ana: «D já foi encontrado. Podemos terminar com custo 10.»

Bruno: «Devemos escolher C, porque C → D é a ligação mais barata.»

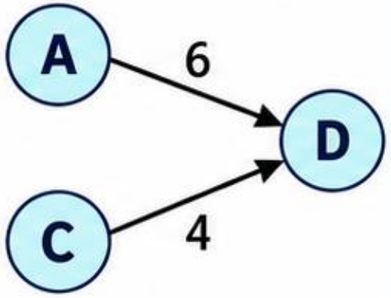
Carla: «Devemos retirar A, embora C possa conduzir a uma solução melhor.»

1. **Quem está correto? Justifique usando a regra de seleção da UCS.**
2. Mostre a evolução de OPEN até confirmar D. Indique as alterações do custo e do pai de D.
3. Explique por que explorar A primeiro não impede encontrar a solução mais barata.

UCS: encontrar o objetivo basta para terminar?

QUESTÃO

Numa execução da UCS, D é o objetivo.
Os custos dos arcos são não negativos.



OPEN = [A:3, C:4, D:10]

A:**3** significa que o melhor caminho conhecido desde o início até A custa 3.

Nas próximas expansões, considerar apenas estas duas ligações.
D não tem sucessores.

Ana: «D já foi encontrado. Podemos terminar com custo 10.»

Bruno: «Devemos escolher C, porque C → D é a ligação mais barata.»

Carla: «Devemos retirar A, embora C possa conduzir a uma solução melhor.»

1. Quem está correto? Justifique usando a regra de seleção da UCS.
2. Mostre a evolução de OPEN até confirmar D. Indique as alterações do custo e do pai de D.
3. Explique por que explorar A primeiro não impede encontrar a solução mais barata.

SOLUÇÃO

1 Escolher pelo custo acumulado

Carla está correta: A:3 tem o menor custo acumulado em OPEN.

Ana confunde gerar o objetivo com confirmar a solução ótima.
Bruno compara apenas o custo da última ligação.

2 Atualizar a fronteira e os pais

Operação	OPEN após o passo	Atualização de D
Expandir **A:3**	[C:4, D:9]	3 + 6 = 9 < 10; pai(D) = A
Expandir **C:4**	[D:**8**]	4 + 4 = 8 < 9; pai(D) = C
Retirar D:**8**	[]	Confirmar objetivo; não expandir.

3 As alternativas continuam disponíveis

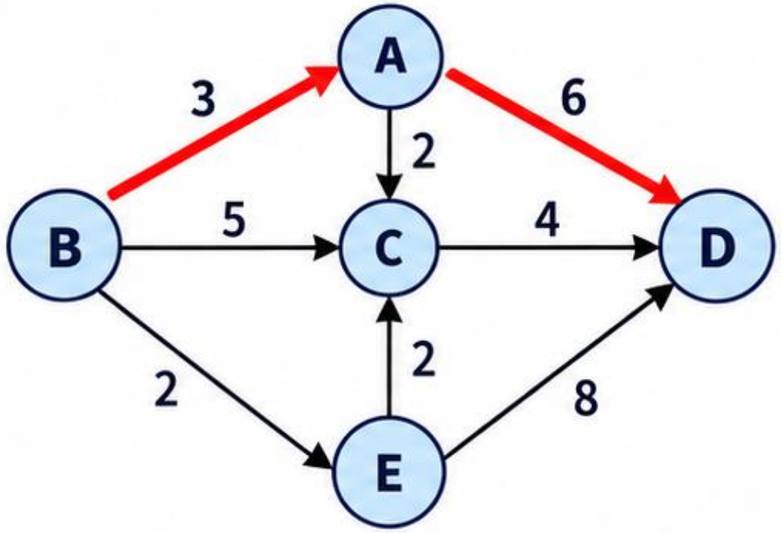
Explorar A não obriga a continuar pelo seu ramo. C permanece em OPEN e é retirado antes de D:9.
Ao explorar C, o custo de D melhora para 8 e o seu pai passa a C.
Reconstrói-se o caminho seguindo os pais; não se usa a ordem de retirada como percurso.

Custo final: 8.
Confirmar D quando é retirado com o menor custo da fronteira.

Verificar uma solução de procura

Uma solução candidata pode ser válida sem ser ótima.

1. Problema



Início: B com 12 unidades de energia

Objetivo: chegar a D com energia ≥ 2

2. A solução é válida?



- ✓ **Ligações permitidas:** sim
- ✓ **Energia suficiente em cada ação:** sim
- ✓ **Chega a D:** sim
- ✓ **Energia final ≥ 2:** sim, termina com 3

B → A → D é válida

Custo: 3 + 6 = 9

3. É ótima em custo?

Caminho	Custo	Energia final
B → A → D	9	3
B → E → C → D	8	4

✗ **B → A → D não é ótimo:** existe uma solução válida com custo inferior.

ⓘ **Para provar que o custo 8 é ótimo,** é necessária uma garantia ou uma verificação adicional.

Para verificar qualquer solução

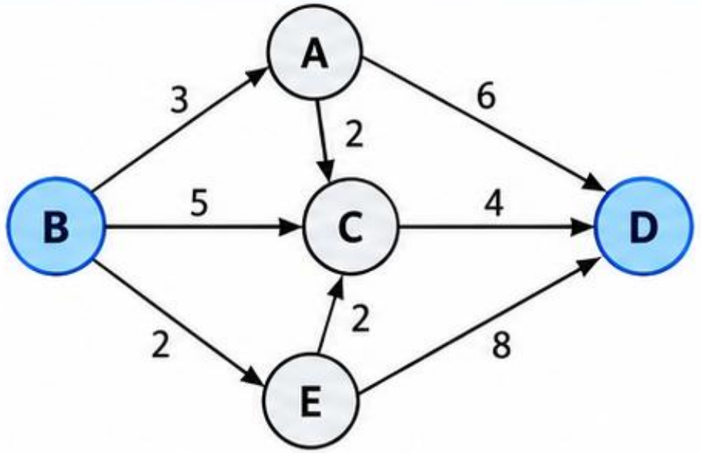
1. Verificar as pré-condições de cada ação.
2. Verificar as restrições durante o percurso e no final.
3. Calcular o custo segundo o critério do problema.
4. Confirmar a garantia ou prova antes de afirmar que é ótima.
5. Confirmar que o modelo representa a tarefa real.

💡 **Uma solução válida pode não ter o menor custo.**

Espaço de estados e alcançabilidade

Que estados podem ser atingidos a partir do estado inicial?

1. O modelo



$S = \{A, B, C, D, E\}$
Estado inicial: B

Modelo simplificado: estado = local; ignoram-se os custos e as restrições de energia.

Espaço de estados S: todos os estados considerados no modelo.
Conjunto alcançável R: estados atingíveis a partir do estado inicial.

2. Construir o conjunto alcançável

Passo	Novos estados	Conjunto acumulado
0	B	$R_0 = \{B\}$
até 1 ação	A, C, E	$R_1 = \{B, A, C, E\}$
até 2 ações	D	$R_2 = \{B, A, C, E, D\}$
passo seguinte	nenhum	$R_3 = R_2$

R_k contém os estados alcançáveis em até k ações; por isso mantém os estados anteriores.

$$R_{k+1} = R_k \cup \text{sucessores}(R_k)$$

Quando não surgem novos estados, atingimos um ponto fixo.

3. Conclusões e algoritmo

```
R ← {B}
Repetir:
    adicionar a R os sucessores dos estados em R
Parar quando não surgirem novos estados
Devolver R
```

- Neste exemplo, $R = S = \{A, B, C, D, E\}$.
- $D \in R$: existe um caminho de B até D no modelo simplificado.
- Se nenhum objetivo pertencer a R, não existe solução nesse modelo.
- Num grafo finito, o processo termina; num espaço infinito, pode não terminar.

O espaço de estados contém os estados do modelo; a alcançabilidade identifica os que são atingíveis a partir do início.

Tempo e memória da procura

1 O que comparamos?

Tempo:
nós gerados ou examinados.

Memória:
nós e estados guardados.

2 Parâmetros

b	número máximo de sucessores de um nó.
d	profundidade da solução mais próxima.
m	profundidade máxima da procura.

3 Comparação na procura em árvore – limites de pior caso

Estratégia	Tempo	Memória da fronteira	Interpretação
BFS	$O(b^{d+1})$	$O(b^{d+1})$	**BFS:** crescimento exponencial com a profundidade.
DFS	$O(b^{m})$	$O(b \times m)$	**DFS:** tempo exponencial; memória proporcional a $b \times m$.

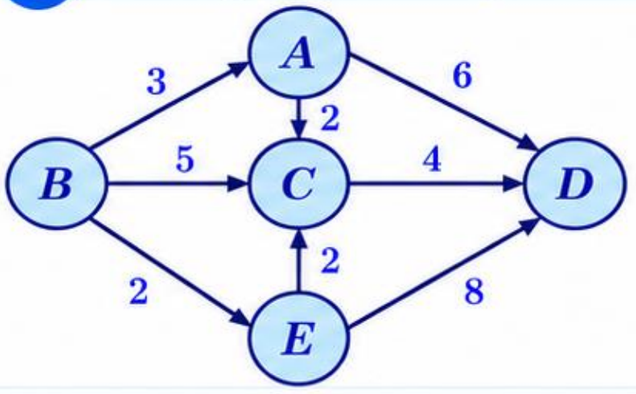
$O(\,)$ indica como os recursos podem crescer no pior caso.
BFS: teste do objetivo ao retirar; **DFS:** m finito.

4 Crescimento: exemplo teórico com $b = 3$

Nível (profundidade)	0	1	2	3
Número de nós	1	3	9	27

Exemplo teórico com $b = 3$.

5 E na procura em grafo?



- Para controlar repetições, guardam-se os estados alcançados, além da fronteira.
- Se o estado for apenas o local, há 5 estados: A, B, C, D e E.
- Se incluir energia, o mesmo local com energias diferentes pode representar estados distintos.

A BFS pode guardar muitos nós; a DFS pode percorrer caminhos longos.
Em grafos, contabilizam-se também os estados alcançados.

Como reconstruir o caminho?

Depois de encontrar o objetivo, como sabemos por onde o robô passou?

Cada nó guarda uma referência ao nó de onde veio: o seu pai.

1 O que guardamos em cada nó?

Campo	Significado
Estado	Onde o robô está
Pai	Nó de onde veio
Custo acumulado	Custo desde o início

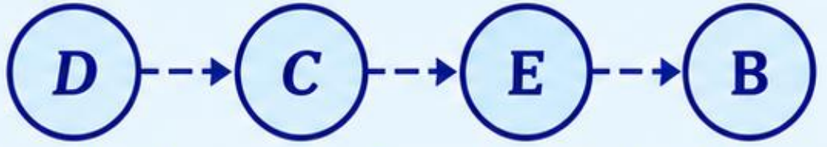
2 Exemplo: registos encontrados

Nó	Pai	Custo acumulado
B	—	0
E	B	2
C	E	4
D	C	8

O pai de D é C: o robô chegou a D a partir de C.

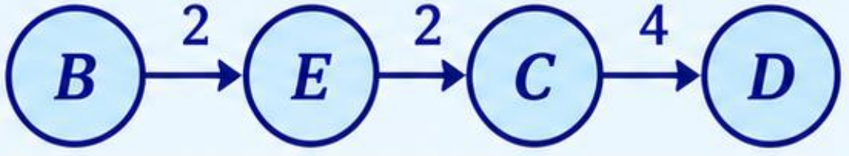
3 Reconstruir o caminho em dois passos

1 Seguir os pais a partir do objetivo



ligações aos pais, não movimentos do robô

2 Inverter a sequência



Custo total: 2 + 2 + 4 = 8

Cada nó guarda de onde veio.

Para recuperar o caminho, seguimos os pais até ao início e invertemos a sequência.

Esta técnica pode ser usada em BFS, DFS e UCS.

Garantias e escolha da estratégia

Menos ações ou menor custo: o que pretende encontrar?

BFS — menos ações

Encontra uma solução com o menor número de passos.

DFS — explorar em profundidade

Não garante menos passos nem menor custo.

UCS — menor custo

Procura a solução com menor custo total.

O mesmo problema, resultados diferentes

Estratégia	Caminho encontrado	Número de ações	Custo total
BFS	B → A → D	2	9
DFS	B → E → D	2	10
UCS	B → E → C → D	3	8

O caminho concreto pode depender da ordem dos sucessores.

Mais ações podem ter menor custo:
3 ações custam 8; 2 ações custam 9.

Garantias num espaço finito com controlo de estados repetidos

Estratégia	Encontra solução?	Menos ações?	Menor custo?
BFS	Sim	Sim	Se custos iguais e positivos
DFS	Sim	Sem garantia	Sem garantia
UCS	Sim	Sem garantia	Sim, com custos não negativos e atualização correta

Sem garantia pode acontecer num caso concreto.

E em espaços infinitos?

DFS: pode ficar presa num ramo infinito.

BFS: completa com ramificação finita e solução a profundidade finita.

UCS: completa com ramificação finita, solução de custo finito e custo por ação $\geq \varepsilon > 0$.

Definições

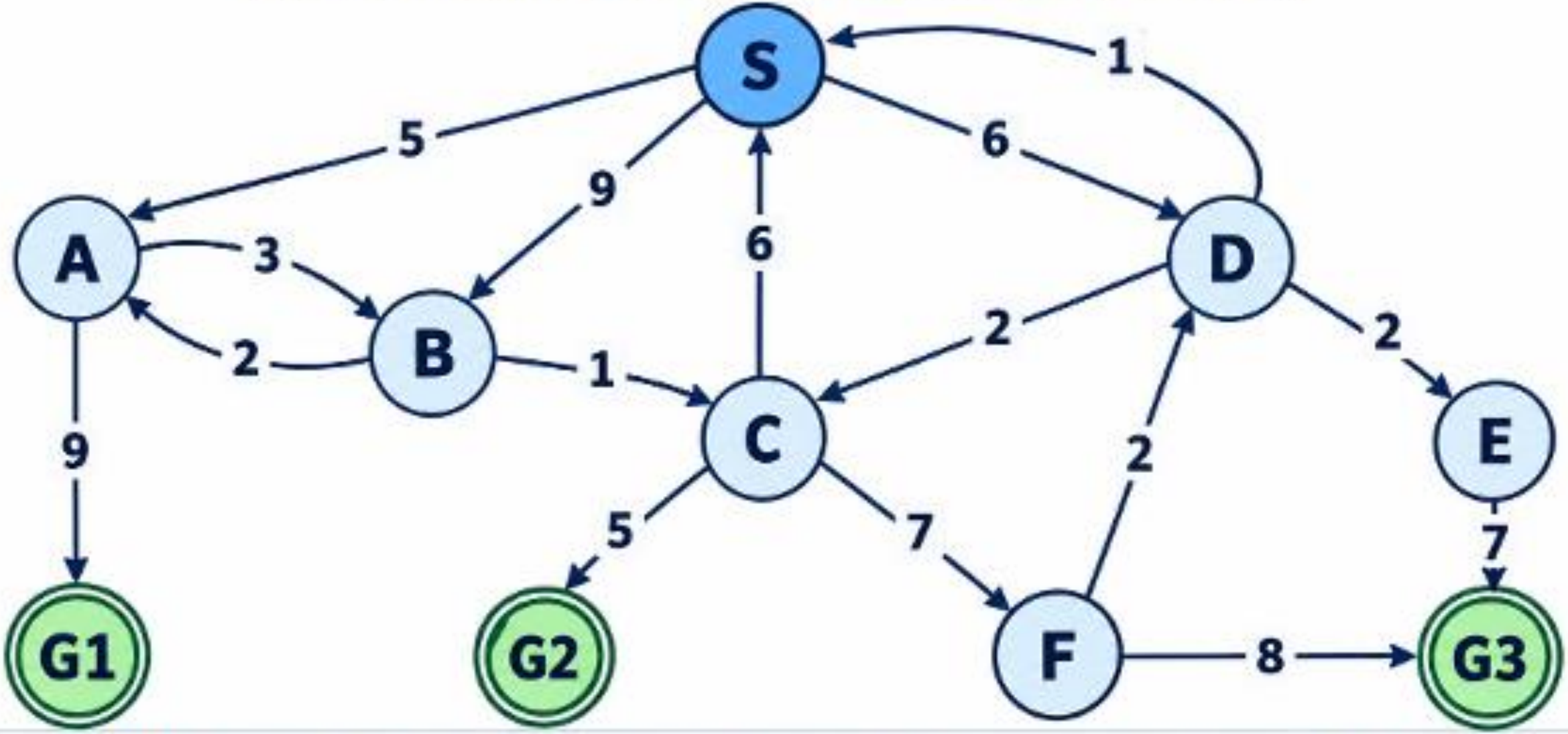
Completude: encontra uma solução, se existir.

Optimalidade em custo: garante uma solução de menor custo.

Menos ações não significa menor custo. Escolha a estratégia segundo o objetivo e as condições do problema.

Grafo de pesquisa — BFS, DFS e UCS

Estado inicial: S | Estados objetivo: G1, G2 e G3



Azul: estado inicial | **Verde:** estados objetivo | **Número no arco:** custo da ação

As setas indicam o sentido permitido. O custo de um caminho é a soma dos custos dos seus arcos.

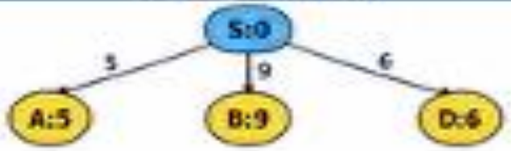
UCS — Construção da árvore em 9 passos

Encontrar o objetivo de menor custo: G1, G2 ou G3

= nó visitado (expandido) | = nó pendente (em OPEN) | = nó descartado (ao sair de OPEN) | = objetivo (encontrado)

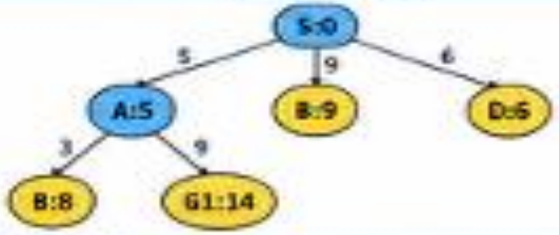
Entradas repetidas são descartadas ao sair de OPEN; um estado visitado não é expandido novamente.
Nos arcos: custo da ação. Nos nós: custo acumulado.

1. Expandir S



Visitados: {S}
Próximo: A:5.

2. Expandir A



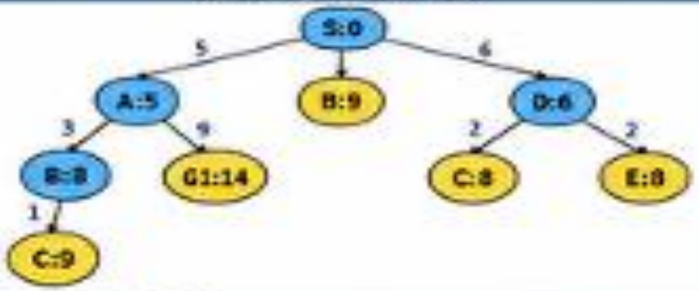
Visitados: {S, A}
Novo caminho para B: 5 + 3 = 8. Próximo: D:6.

3. Expandir D



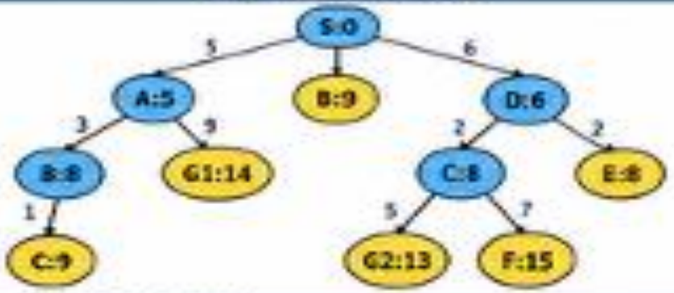
Visitados: {S, A, D}
B, C e E empatam a 8. Escolher B, inserido primeiro.

4. Expandir B:8



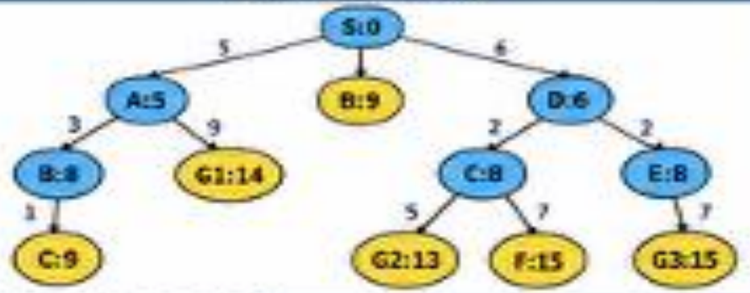
Visitados: {S, A, D, B}
Por B, chegar a C custa 8 + 1 = 9. Próximo: C:8.

5. Expandir C:8



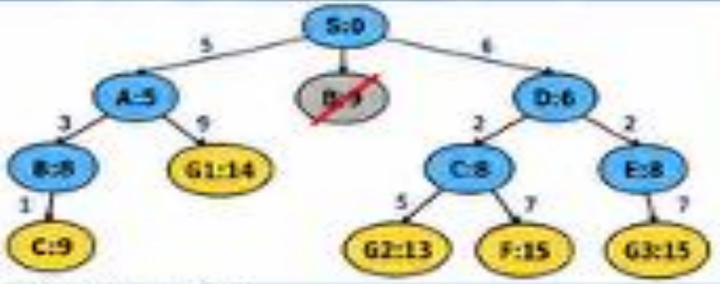
Visitados: {S, A, D, B, C}
Gerar G2:13 e F:15. Próximo: E:8.

6. Expandir E:8



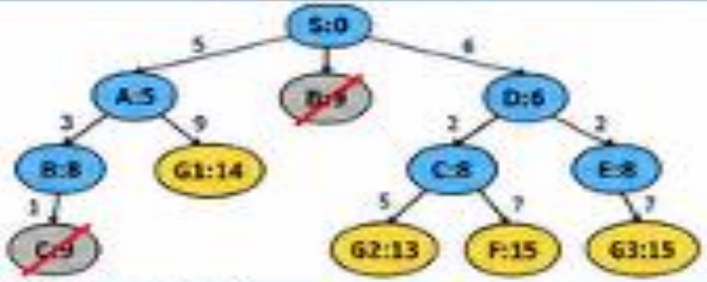
Visitados: {S, A, D, B, C, E}
Os três objetivos foram gerados; ainda não terminar.

7. Descartar B:9



Visitados: {S, A, D, B, C, E}
B já foi visitado com custo 8. Não voltar a expandir B:9.

8. Descartar C:9



Visitados: {S, A, D, B, C, E}
C já foi visitado com custo 8. Não voltar a expandir C:9.

9. Retirar G2:13 e terminar



Visitados: {S, A, D, B, C, E}
Menor custo pendente: G2:13. Confirmar o objetivo.

Solução: S → D → C → G2 | **Custo: 6 + 2 + 5 = 13**

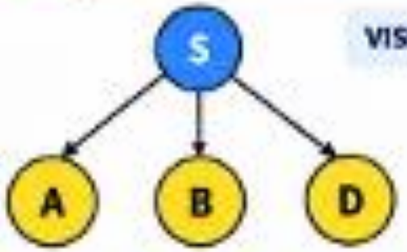
G1:14 e G3:15 são mais caros.
Os regressos a estados já visitados foram omitidos das árvores.

BFS — Construção da árvore passo a passo

Fila FIFO: primeiro a entrar, primeiro a sair.

= expandido | = pendente (na fila) | = entrada repetida descartada | = objetivo confirmado

1. Expandir S

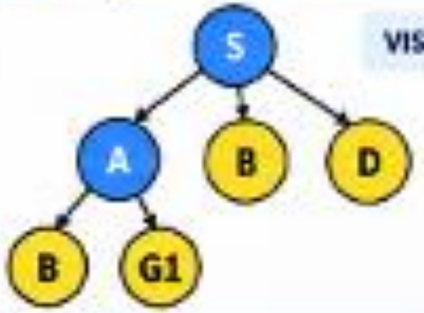


VISITED: S

FILA (próximo à esquerda):

A	B(S)	D

2. Expandir A

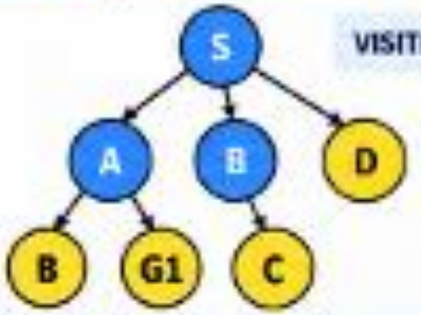


VISITED: S, A

FILA (próximo à esquerda):

B(S)	D	B(A)	G1

3. Expandir B, filho de S

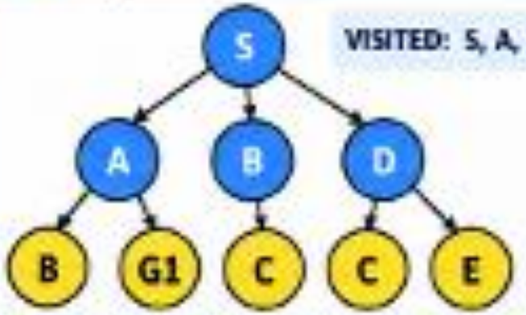


VISITED: S, A, B

FILA (próximo à esquerda):

D	B(A)	G1	C(B)

4. Expandir D

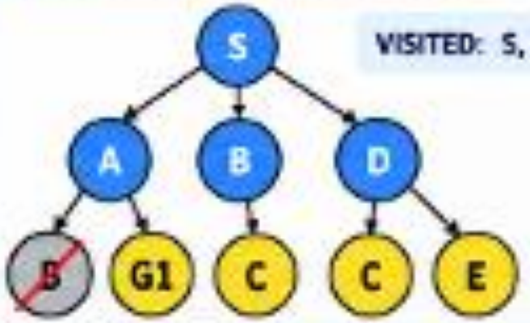


VISITED: S, A, B, D

FILA (próximo à esquerda):

B(A)	G1	C(B)	C(D)	E

5. Descartar B repetido



VISITED: S, A, B, D

FILA (próximo à esquerda):

G1	C(B)	C(D)	E

6. Retirar G1 e terminar



VISITED: S, A, B, D, G1

FILA restante (próximo à esquerda):

C(B)	C(D)	E

7. Reconstruir o caminho e calcular o custo



Caminho: S → A → G1

Profundidade: 2 ações

Custo: 5 + 9 = 14

A BFS minimiza o número de ações; não necessariamente o custo.

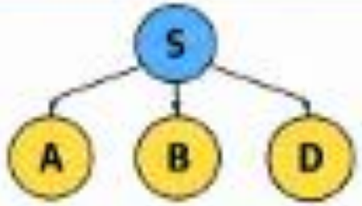
Encontra um objetivo com o menor número de ações, não garante o menor custo quando as ações têm custos diferentes.

Próximo a retirar: à esquerda da fila. **B(A):** entrada B com pai A. Repetidos são descartados na retirada se já visitados.

DFS — Construção da árvore passo a passo

Pilha LIFO: último a entrar, primeiro a sair

1. Expandir S

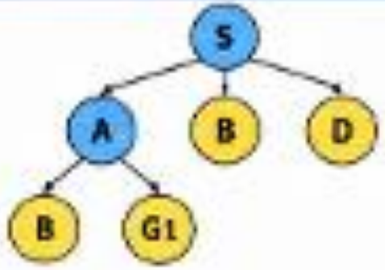


PILHA — topo à esquerda

A	B(S)	D(S)

VISITED: { S }

2. Expandir A

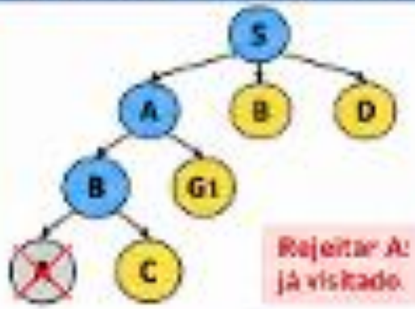


PILHA — topo à esquerda

B(A)	G1	B(S)	D(S)

VISITED: { S, A }

3. Expandir B, filho de A



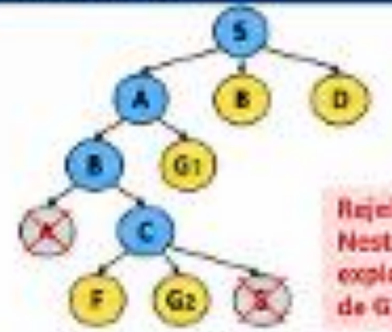
Rejeitar A: já visitado.

PILHA — topo à esquerda

C	G1	B(S)	D(S)

VISITED: { S, A, B }

4. Expandir C



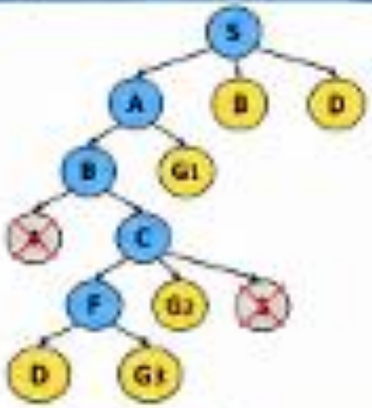
Rejeitar S. Nesta ordem, explorar F antes de G2.

PILHA — topo à esquerda

F	G2	G1	B(S)	D(S)

VISITED: { S, A, B, C }

5. Expandir F

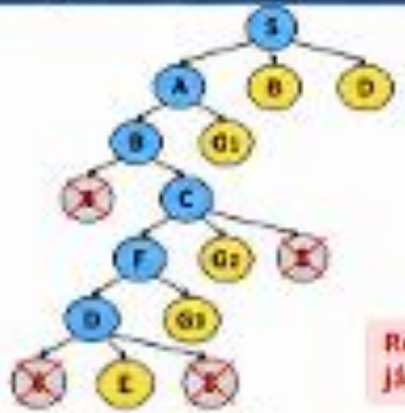


PILHA — topo à esquerda

D(F)	G3(F)	G2	G1	B(S)	D(S)

VISITED: { S, A, B, C, F }

6. Expandir D, filho de F



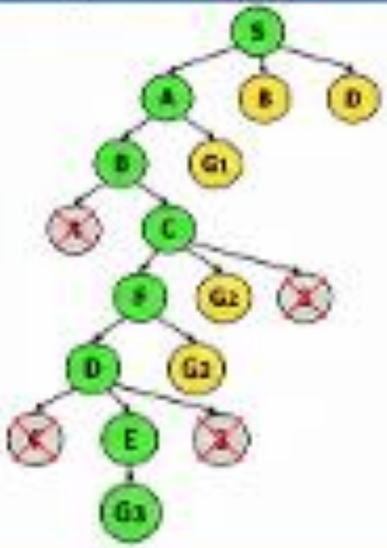
Rejeitar C e S: já visitados.

PILHA — topo à esquerda

E	G3(F)	G2	G1	B(S)	D(S)

VISITED: { S, A, B, C, F, D }

7. Expandir E e retirar G3



Após expandir E:

G3(E)	G3(F)	G2	G1	B(S)	D(S)

Retirar G3(E): objetivo confirmado.

Pilha restante:

G3(F)	G2	G1	B(S)	D(S)

Resultado:
S → A → B → C → F → D → E → G3
Profundidade: 7 ações
Custo: 5 + 3 + 1 + 7 + 2 + 2 + 7 = 27

VISITED: { S, A, B, C, F, D, E, G3 }

A DFS depende da ordem dos sucessores; não garante menor profundidade nem menor custo.
B(A): entrada B gerada por A. Para explorar primeiro o filho da esquerda, inserem-se os filhos na pilha pela ordem inversa.
Múltiplas ocorrências representam o mesmo estado através de caminhos diferentes.

Estado expandido (já visitado) · Estado pendente (na pilha) · Sucessor rejeitado (já visitado) · Caminho solução / objetivo